

## **Introduction**

### **1. About the workbook:**

This workbook is intended to be used by T.Y.B.SC (Computer Science) students for the CS-367 –Operating system Laboratory Assignments in Semester–VI. This workbook is designed by considering all the practical concepts topics mentioned in syllabus.

### **2. The objectives of this workbook are:**

- 1) Defining the scope of the course.
- 2) To have continuous assessment of the course and students.
- 3) Providing ready reference for the students during practical implementation.
- 4) Provide more options to students so that they can have good practice before facing the examination
- 5) Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

### **3. How to use this workbook:**

- 1) The workbook is divided into five assignments.
- 2) Each OS assignment has three SETs.
- 3) It is mandatory for students to complete SETA, SETB and SET C in given slot.

### **4. Instructions to the students**

- 1) Students are expected to carry this book every time when they come to the lab for practical.
- 2) Students should prepare themselves before hand for the Assignment by reading these materials.
- 3) Students should prepare themselves before hand for the Assignment by reading these materials.
- 4) Instructor will specify which problems to solve in the lab during the allotted slot and student should complete the verified by the instructor .How every student should spend additional hours in Lab and at home to cover as many problems as possible given in this workbook.
- 5) Students will be assessed for each exercise on as scale from 0 to5.

|                   |   |
|-------------------|---|
| Not done          | 0 |
| Incomplete        | 1 |
| Late Complete     | 2 |
| Needs improvement | 3 |
| Complete          | 4 |
| Well Done         | 5 |

## 5. Guide lines for Instructors

- 1) Explain the assignment and related concepts in around ten minute using white board if required or by demonstrating on Projector.
- 2) You should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- 3) The value should also been written on assignment completion page of the respective Lab course.

## 6. Guide lines for Lab Administrator

You have to ensure appropriate hardware and software is available to each student. The operating system and software requirement so server side and also client side areas given below:

- 1) Operating System-Linux (Ubuntu/Fedora/Centos)
- 2) Editor: Any linux based editor like vi, gedit, eclipse etc.
- 3) Jupyter notebook or any IDLE of python on any platform.

### Index

| Assignmet No | Assignment Name                                                                         | Page Number |
|--------------|-----------------------------------------------------------------------------------------|-------------|
| 1            | Simulation of Banker's Algorithm of Deadlock Avoidance in processes of operating system |             |
| 2            | Simulation of File Allocation Methods and free space management                         |             |
| 3            | Simulation of Disk Scheduling Algorithms                                                |             |
| 4            | Assignment based on distributed and mobile OS                                           |             |

## Assignment Completion Sheet

| Subject: USCSP- 367 Laboratory(Operating System-II) |                                                                                         |                  |                |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------|------------------|----------------|
| Sr. No.                                             | Assignment Name                                                                         | Marks (Out of 5) | Teacher's Sign |
| 1                                                   | Simulation of Banker's Algorithm of Deadlock Avoidance in processes of operating system |                  |                |
| 2                                                   | Simulation of File Allocation Methods and free space management                         |                  |                |
| 3                                                   | Simulation of Disk Scheduling Algorithms                                                |                  |                |
| 4                                                   | Assignment based on distributed and mobile OS                                           |                  |                |
| Total Out of : 20                                   |                                                                                         |                  |                |
| Total (Out of 10)                                   |                                                                                         |                  |                |

This is to certify that Mr. /Ms. \_\_\_\_\_ has successfully completed the course work for USCSP 367: **Operating Systems-II** Laboratory and has Scored \_\_\_\_ marks out of 10.

**Instructor**

**Internal Examiner**

**External Examiner**

## ***CERTIFICATE***

This is to certify that Mr./Ms.\_\_\_\_\_

has successfully completed **TYBSC(Computer Science)**

**USCSP-367 Practical course in Operating Systems-II Semester VI course**

in the year\_\_\_\_\_and

his/her seat number is\_\_\_\_\_.

He/She has scored marks\_\_out of 10.

Instructor

H.O.D./Coordinator

***Internal Examiner***

***External Examiner***

## Assignment No: 1 Deadlock Detection and Avoidance

Every process needs some resources to complete its execution. However, the resource is granted in a sequential order.

1. The process requests for some resource.
2. OS grant the resource if it is available otherwise let the process waits.
3. The process uses it and release on the completion.

The process uses it and release on the completion.

A Deadlock is a situation where each of the computer process waits for a resource which is being assigned to some another process. In this situation, none of the process gets executed since the resource it needs, is held by some other process which is also waiting for some other resource to be released.

Let us assume that there are three processes P1, P2 and P3. There are three different resources R1, R2 and R3. R1 is assigned to P1, R2 is assigned to P2 and R3 is assigned to P3. After some time, P1 demands for R2 which is being used by P2. P1 halts its execution since it can't complete without R2. P2 also demands for R3 which is being used by P3. P2 also stops its execution because it can't continue without R3. P3 also demands for R1 which is being used by P1 therefore P3 also stops its execution. So, this condition is called as deadlock.

Deadlock Characterization: Necessary Conditions:

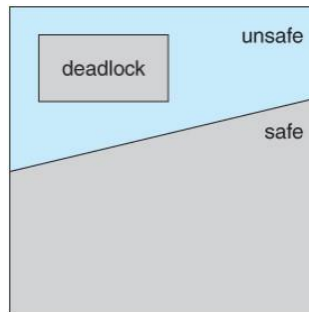
- Mutual Exclusion: A resource can only be shared in mutually exclusive manner. It implies, if two processes cannot use the same resource at the same time.
- Hold and Wait: A process waits for some resources while holding another resource at the same time.
- No preemption: The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile.
- Circular Wait: All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.

Deadlock Avoidance - Safe state, Resource-Allocation-Graph Algorithm, Banker's Algorithm

### Safe State

- A state is **safe** if the system can allocate all resources requested by all processes (up to their stated maximums) without entering a deadlock state.
- More formally, a state is safe if there exists a safe sequence of processes  $\{P_0, P_1, P_2, \dots, P_N\}$  such that all of the resource requests for  $P_i$  can be granted using the resources currently allocated to  $P_i$  and all processes  $P_j$  where  $j < i$ . (i.e. if all the processes prior to  $P_i$  finish and free up their resources, then  $P_i$  will be able to finish also, using the resources that they have freed up.)

- If a safe sequence does not exist, then the system is in an unsafe state, which may lead to deadlock. (All safe states are deadlock free, but not all unsafe states lead to deadlocks.)



**Figure: Safe, Unsafe and deadlocked state spaces.**

- For example, consider a system with 12 tape drives, allocated as follows. Is this a safe state? What is the safe sequence?

|           | Maximum Needs | Current Allocation |
|-----------|---------------|--------------------|
| <b>P0</b> | 10            | 5                  |
| <b>P1</b> | 4             | 2                  |
| <b>P2</b> | 9             | 2                  |

- What happens to the above table if process P2 requests and is granted one more tape drive?
- Key to the safe state approach is that when a request is made for resources, the request is granted only if the resulting allocation state is a safe one.

### Deadlock Avoidance

Deadlock avoidance can be done with Banker's Algorithm.

### Banker's Algorithm

Banker's Algorithm is resource allocation and deadlock avoidance algorithm which test the entire request made by processes for resources, it checks for the safe state, if after granting request system remains in the safe state it allows the request and if there is no safe state it doesn't allow the request made by the process.

#### Inputs to Banker's Algorithm:

1. Max need of resources by each process.
2. Currently, allocated resources by each process.
3. Max free available resources in the system.

The request will only be granted under the below condition:

1. If the request made by the process is less than equal to max need to that process.
2. If the request made by the process is less than equal to the freely available resource in the system.

Several data structures are used to implement Banker's algorithm. Let  $n$  be number of processes and  $m$  be number of resource types.

1. **Available:** A vector of length  $m$  indicating the number of available resources of each type. If  $\text{Available}[j] = k$  means there are  $k$  instances of resource type  $R_j$  available.
2. **Max:** An  $n \times m$  matrix defining the maximum demand of each process. If  $\text{Max}[i, j] = k$ , then process  $P_i$  may request at most  $k$  instances of resource type  $R_j$ .
3. **Allocation:** An  $n \times m$  matrix defining number of resources of each type currently allocated to each process. If  $\text{Allocation}[i, j] = k$  then process  $P_i$  is currently allocated  $k$  instances of resource type  $R_j$ .
4. **Need:** An  $n \times m$  matrix indicating the remaining resource need of each process. If  $\text{Need}[i, j] = k$ , then process  $P_i$  may need  $k$  more instances of resource type  $R_j$ , in order to complete its task.

### Safety Algorithm

Let  $\text{Work}$  and  $\text{Finish}$  be vectors of length  $m$  and  $n$ , respectively

1. Initialize  $\text{Work} = \text{Available}$
2.  $\text{Finish}[i] = \text{false}$  for  $i = 0, 1, 2, \dots, n - 1$ .
3. Find  $i$  such that both:
  - a)  $\text{Finish}[i] = \text{false}$
  - b)  $\text{Need}_i \leq \text{Work}$
4. If no such  $i$  exists, go to step 6.
5.  $\text{Work} = \text{Work} + \text{Allocation}_i$ ,  $\text{Finish}[i] = \text{true}$ , go to step 2.
6. If  $\text{Finish}[i] = \text{true}$  for all  $i$ , then the system is in a safe state.

### Resource – Request Algorithm

- This algorithm determines if requests can be safely granted.
- Let  $\text{Request}$  be request vector for process  $P_i$ . If  $\text{Request}_i[j] = k$  then process  $P_i$  wants  $k$  instances of resource type  $R_j$ .
  1. If  $\text{Request}_i \leq \text{Need}_i$  go to step 2. Otherwise, raise error condition, since Process has exceeded its maximum claim.
  2. If  $\text{Request}_i \leq \text{Available}$ , go to step 3. Otherwise  $P_i$  must wait, since resources are not available.
  3. Pretend to allocate requested resources to  $P_i$  by modifying the state as follows:
    - a)  $\text{Available} = \text{Available} - \text{Request}_i$ ;
    - b)  $\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i$
    - c)  $\text{Need}_i = \text{Need}_i - \text{Request}_i$

After applying this algorithm,

- If safe state  $\Rightarrow$  the resources are allocated to  $P_i$ .

- If unsafe state  $\Rightarrow P_i$  must wait, and the old resource-allocation state is restored

#### **ALGORITHM:**

1. Start the program.
2. Get the values of resources and processes.
3. Get the available value.
4. After allocation find the need value.
5. Check whether it's possible to allocate
6. If it is possible then the system is in safe state.
7. Else system is not in safety state.
8. If the new request comes then check that the system is in safety.
9. If we allow the request.
10. Stop the program.

#### **DEADLOCK DETECTION**

- Deadlock detection detects a deadlock by checking whether all conditions necessary for a deadlock hold simultaneously.
- The systems that do not implement algorithms for deadlock prevention or avoidance must implement an algorithm for deadlock detection and recovery.
- The system must maintain information about:
  - Process status: resource allocated and requested by a process.
  - An algorithm to detect the deadlock.
- There are two algorithms i.e. Single instance resource type and several instances of a resource type.
- The deadlock detection algorithm uses same data structures as Banker's algorithm:
  - a) Available = A vector of length m indicating number of available resources of each type.
  - b) Allocation = An  $n \times m$  matrix indicating the number of resources of each type currently allocated to each process.
  - c) Request = A  $n \times m$  matrix indicating the current request of each process. If  $\text{Request}[i][j] = k$  then process  $P_i$  is requesting k more instances of resource type  $R_j$ .

#### **ALGORITHM:**

**Step 1:** Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available. For  $i = 1, 2, \dots, n$ , if  $\text{Allocation}_i \neq 0$ , then  $\text{Finish}[i] = \text{false}$ ; otherwise,  $\text{Finish}[i] = \text{true}$ .

**Step 2:** Find an index i such that both:

- a)  $\text{Finish}[i] == \text{false}$
- b)  $\text{Request}_i \leq \text{Work}$

If no such i exists, go to step 4.

**Step 3:**  $\text{Work} = \text{Work} + \text{Allocation}_i$ ,  $\text{Finish}[i] = \text{true}$  go to step 2.

**Step 4:** If  $\text{Finish}[i] == \text{false}$ , for some i,  $1 \leq i \leq n$ , then the system is in deadlock state. Moreover, if  $\text{Finish}[i] == \text{false}$ , then  $P_i$  is deadlocked.

Algorithm requires an order of  $O(m \times n^2)$  operations to detect whether the system is in deadlock state.



## Practical Assignments:

### Set A

- Partially implement the Menu driven Banker's algorithm for accepting Resource Allocation, Max from user.
  - Accept Total Resource
  - Display Allocation, Max
  - Find Need matrix and Display It,
  - Display Available
- Consider the system with 3 resources types A,B and C with 10,7,8 instances respectively. Consider the following snapshot:

|                | Allocation |   |   |
|----------------|------------|---|---|
|                | A          | B | C |
| P <sub>0</sub> | 0          | 1 | 0 |
| P <sub>1</sub> | 2          | 0 | 0 |
| P <sub>2</sub> | 3          | 0 | 3 |
| P <sub>3</sub> | 2          | 1 | 1 |
| P <sub>4</sub> | 0          | 0 | 2 |

|                | Request |   |   |
|----------------|---------|---|---|
|                | A       | B | C |
| P <sub>0</sub> | 0       | 1 | 0 |
| P <sub>1</sub> | 2       | 2 | 2 |
| P <sub>2</sub> | 3       | 4 | 4 |
| P <sub>3</sub> | 2       | 5 | 1 |
| P <sub>4</sub> | 0       | 0 | 2 |

| Available |   |   |
|-----------|---|---|
| A         | B | C |
| 3         | 5 | 2 |

Answer the following questions:

- Display the contents of Need array?
- Is there any deadlock? Print the message.

### Set B

- Consider the following snapshot of system A,B,C and D are the resource type

|                | Allocation |   |   |   |                | MAX |   |   |   |  | Available |    |    |    |
|----------------|------------|---|---|---|----------------|-----|---|---|---|--|-----------|----|----|----|
|                | A          | B | C | D |                | A   | B | C | D |  | A         | B  | C  | D  |
| P <sub>0</sub> | 0          | 6 | 3 | 2 | P <sub>0</sub> | 0   | 6 | 5 | 2 |  | 3         | 14 | 12 | 12 |
| P <sub>1</sub> | 0          | 0 | 1 | 2 | P <sub>1</sub> | 0   | 0 | 1 | 2 |  |           |    |    |    |
| P <sub>2</sub> | 1          | 0 | 0 | 0 | P <sub>2</sub> | 1   | 7 | 5 | 0 |  |           |    |    |    |
| P <sub>3</sub> | 1          | 3 | 5 | 4 | P <sub>3</sub> | 2   | 3 | 5 | 6 |  |           |    |    |    |
| P <sub>4</sub> | 0          | 0 | 1 | 4 | P <sub>4</sub> | 0   | 6 | 5 | 6 |  |           |    |    |    |

- Calculate and display the content of need matrix?
- Is the system in safe state? If display the safe sequence.
- If a request from process P arrives for (0, 0, 4, 1) can it be granted immediately by keeping the system in safe state. Print a message

### Assignment Evaluation

0: Not Done [ ]

1: Incomplete [ ]

2: Late Complete [ ]

3: Needs Improvement [ ]

4: Complete [ ]

5: Well done [ ]

Signature of the instructor: \_\_\_\_\_ Date: \_\_\_\_\_

## Assignment No.2: File Allocation Methods

Files are managed by the operating system on secondary storage devices. Hard disk drive is most commonly used storage device.

Physically hard disk is divided into number sectors also called as blocks. When a file is to be created the free blocks on the disk are allocated to the file.

Operating system maintains the status of each block. Status of disk block can be

- 1) Free
- 2) Allocated
- 3) Reserved
- 4) Bad

Purpose of this practical assignment is to understand various file allocation methods and their implementation.

When user accesses any file, the system has to locate all the disk blocks of that file on disk.

Hence operating system has systematic way to maintain, allocate the blocks for each file and releasing the blocks of file when it is deleted.

File Allocation Method is a method that specifies how the blocks allocated to file are maintained so that file can be accessed properly. Most commonly used file allocation methods are

- 1) Contiguous File Allocation
- 2) Linked File Allocation
- 3) Indexed File Allocation

### **1. Contiguous File Allocation:**

- ☐ This file allocation method is also called as Sequential file allocation.
- ☐ In this method blocks allocated the file are contiguous. That is if file is of  $n$  blocks then all the  $n$  blocks are adjacent to one another.
- ☐ Advantage of this method is that the file can access sequentially as well as randomly.
- ☐ Directory maintains the file name along with starting block number and length (that is number of blocks allocated to file).
- ☐ Drawback of this method is that there can be external fragmentation problem and file cannot grow efficiently.

### **2. Linked File Allocation:**

- ☐ In this method  $n$  blocks file are maintained as chain (linked list) of  $n$  blocks. First block of file contains the address or link to second block of file. Second block of file contains the address or link to third of file as so on. Last block of file points to null. That any free block on the storage device can be allocated to the file.
- ☐ Advantage of this method is that the file can grow or shrink dynamically and there is no external fragmentation.
- ☐ Directory maintains the file name along with starting block number and last block number of a file.

- ☐ Drawback of this method is that the file can be access sequentially only. Random file accessing is not possible and wastage of memory for pointer in each block of a file.
- ☐ This method overcomes the drawbacks of Sequential File Allocation method as well as

### **3. Indexed Allocation .**

- ☐ In this method a special Index Block is maintained for each file. This block contains the list of block numbers allocated to the file. When file is to be accessed the index block will give the list of blocks allocated to that file.
- ☐ Hence in this method file can be accessed sequentially as well as randomly.
- ☐ Each available free block on disk can be allocated to any file on demand and that number is added to the index block of that file.
- ☐ Directory maintains the file name along with its index block number and number of entries in it.
- ☐ Drawback of this method is the overhead of maintaining index block.

### **Data Structures:**

#### **1) Bit Vector :**

It is used to maintain the free space on disk. It is a vector of size n (where n is number of blocks on disk) with values 0 or 1. These values mark the status of block as whether it is free or allocated. For Ex. It bit vector is:00111010001101.....It means that first 2 blocks are allocated, block 3,4,5 are free, 6th is allocated, 7th free, 8th to 10th are not free, 11th and 12th are free and so on.

#### **2) Directory :**

Each directory entry maintains the filename along with certain details relevant to file allocation method. Directory list can be maintained statically or dynamically. For Contiguous allocation each directory entry contains filename, starting block number and length. Likewise for each file allocation method appropriate details are maintained in directory entry.

#### **Implementation:**

- ☐ A bit vector (array) of size n which is initialized randomly to 0 or 1.
- ☐ A menu with the options
  - Show Bit Vector
  - Create New File
  - Show Directory
  - Delete File
  - Exit
- ☐ When Create New File option is selected the program should ask the file name along with number of blocks to allocate to the file. According the free blocks should be searched by using the bit vector and allocated to the file. A new directory entry shall be added in directory list along with appropriate details.
- ☐ When Delete File option is selected the program should ask the file name. Release the blocks allocated to the file. Accordingly update the bit vector and remove the entry from directory.

**Practical Assignments:**  
**Set A**

1. Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option

**Set B**

2. Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

**Set C**

3. Write a program to simulate Indexed file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

**Assignment Evaluation**

0: Not Done [ ]

1: Incomplete [ ]

2: Late Complete [ ]

3: Needs Improvement [ ]

4: Complete [ ]

5: Well done [ ]

Signature of the instructor: \_\_\_\_\_ Date: \_\_\_\_\_

## Assignment No.3: Disk Scheduling Algorithms

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, meeting this responsibility entails having fast access time and large disk bandwidth. Both the access time and the bandwidth can be improved by managing the order in which disk I/O requests are serviced which is called as disk scheduling. The simplest form of disk scheduling is, of course, the first - come, first - served (FCFS) algorithm. This algorithm is intrinsically fair, but it generally does not provide the fastest service. In the SCAN algorithm, the disk arm starts at one end, and moves towards the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk. At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk. C - SCAN is a variant of SCAN designed to provide a more uniform wait time. Like SCAN, C - SCAN moves the head from one end of the disk to the other, servicing requests along the way. When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip

### DISK SCHEDULING ALGORITHM

- ☐ The algorithms used for disk scheduling are called as disk scheduling algorithms.
- ☐ The purpose of disk scheduling algorithms is to reduce the total seek time.

#### **1.First Come First Serve (FCFS)**

FCFS is the simplest disk scheduling algorithm. As the name suggests, this algorithm entertains requests in the order they arrive in the disk queue. The algorithm looks very fair and there is no starvation (all requests are serviced sequentially) but generally, it does not provide the fastest service.

#### Algorithm: Advantages-

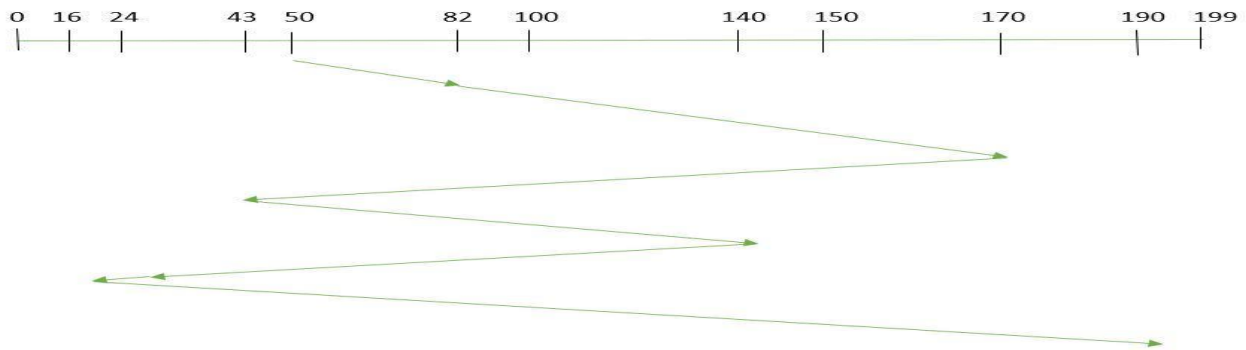
- ☐ It is simple, easy to understand and implement.
- ☐ It does not cause starvation to any request.

#### Disadvantages-

- ☐ It results in increased total seek time.
- ☐ It is inefficient.

#### Example

a) Consider a disk queue with requests for I/O to blocks on cylinders 82,170,43,140,24,16,160. The FCFS scheduling algorithm is used. The head is initially at cylinder number 50. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.



### Example:

Suppose the order of request is-(82,170,43,140,24,16,160)

And current position of Read/Write head is: 50

So, total overhead movement (total distance covered by the disk arm) =  $(82-50)+(170-82)+(170-43)+(140-43)+(140-24)+(24-16)+(190-16) = 642$

## 2. SSTF Disk Scheduling Algorithm-

SSTF stands for Shortest Seek Time First.

- ☐ This algorithm services that request next which requires least number of head movements from its current position regardless of the direction.
- ☐ It breaks the tie in the direction of head movement

### Advantages-

It reduces the total seek time as compared to FCFS.

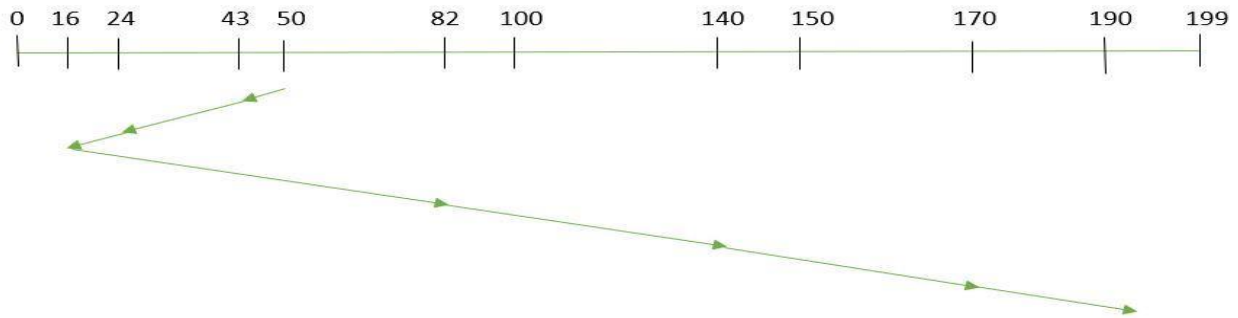
- ☐ It provides increased throughput.
- ☐ It provides less average response time and waiting time.

### Disadvantages-

There is an overhead of finding out the closest request.

- ☐ The requests which are far from the head might starve for the CPU.
- ☐ It provides high variance in response time and waiting time.
- ☐ Switching the direction of head frequently slows down the algorithm.

b)Consider a disk queue with requests for I/O to blocks on cylinders 82,170,43,140,24,16,190. The SSTF scheduling algorithm is used. The head is initially at cylinder number 50 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.



Suppose the order of request is- (82,170,43,140,24,16,190)

And current position of Read/Write head is: 50

total overhead movement (total distance covered by the disk arm) =  
 $(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) = 208$

## SCAN DISK SCHEDULING ALGORITHM

Given an array of disk track numbers and initial head position, our aim is to find the total number of seek operations done to access all the requested tracks if SCAN disk scheduling algorithm is used.

### 3.SCAN (Elevator) algorithm

In SCAN disk scheduling algorithm, head starts from one end of the disk and moves towards the other end, servicing requests in between one by one and reach the other end. Then the direction of the head is reversed and the process continues as head continuously scan back and forth to access the disk. So, this algorithm works as an elevator and hence also known as the elevator algorithm. As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

Advantages of SCAN (Elevator) algorithm

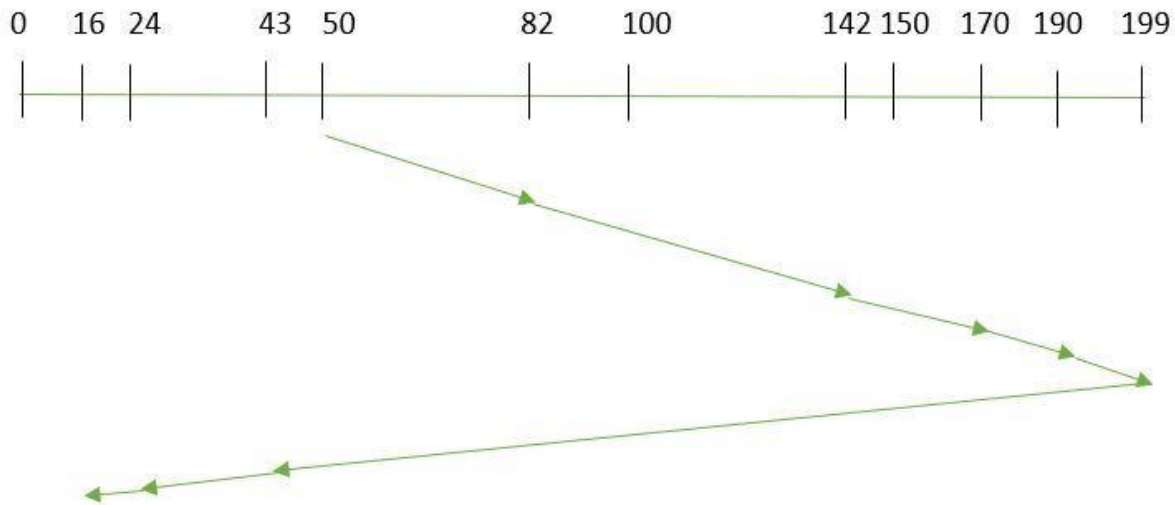
- ☐ This algorithm is simple and easy to understand.
- ☐ SCAN algorithm has no starvation. 3. This algorithm is better than FCFS Scheduling algorithm.

Disadvantages of SCAN (Elevator) algorithm

- ☐ More complex algorithm to implement.
- ☐ This algorithm is not fair because it cause long waiting time for the cylinders just visited by the head.
- ☐ It causes the head to move till the end of the disk in this way the requests arriving ahead of the arm position would get immediate service but some other requests that arrive behind the arm position will have to wait for the request to complete.

c) Consider a disk queue with requests for I/O to blocks on cylinders 82,170,43,140,24,16,190. The SCAN scheduling algorithm is used. The head is initially at cylinder number 50 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move “**towards the larger value**”.



Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as  

$$= (199 - 50) + (199 - 16) = 332$$

#### 4.C-SCAN

- ☐ Circular-SCAN Algorithm is an improved version of the SCAN Algorithm.
- ☐ Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- ☐ After reaching the other end, head reverses its direction.
- ☐ It then returns to the starting end without servicing any request in between.
- ☐ The same process repeats.

#### Advantages-

- ☐ The waiting time for the cylinders just visited by the head is reduced as compared to the SCAN Algorithm.
- ☐ It provides uniform waiting time.
- ☐ It provides better response time.

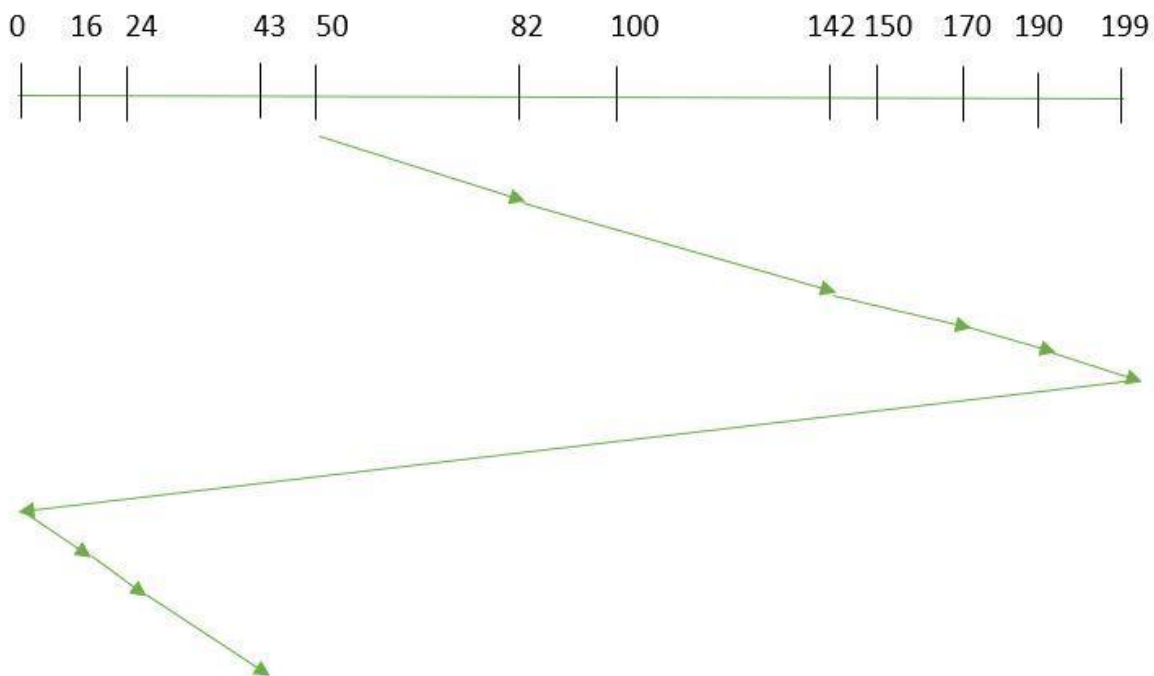


#### Disadvantages-

- It causes more seek movements as compared to SCAN Algorithm.
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

Consider a disk queue with requests for I/O to blocks on cylinders 82,170,43,140,24,16,190. The SCAN scheduling algorithm is used. The head is initially at cylinder number 50 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is \_\_\_\_\_.

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **“towards the larger value”**.



So, the total overhead movement (total distance covered by the disk arm) is calculated as:  
$$=(199-50) + (199-0) + (43-0) = 391$$

**Practical Assignments:****Set A**

- 1) Write an OS program to implement FCFS Disk Scheduling algorithm.
- 2) Write an OS program to implement SSTF algorithm Disk Scheduling algorithm.

**Set B**

- 1) Write an OS program to implement SCAN Disk Scheduling algorithm.
- 2) Write an OS program to implement C-SCAN algorithm Disk Scheduling algorithm.

**Assignment Evaluation**

0: Not Done [ ]

3: Needs Improvement [ ]

1: Incomplete [ ]

4: Complete [ ]

2: Late Complete [ ]

5: Well done [ ]

Signature of the instructor: \_\_\_\_\_ Date: \_\_\_\_\_

## Assignment No. 4:

### Distributed and mobile OS

A distributed system consists of a collection of autonomous computers, connected through a network and distribution middleware, which enables computers to coordinate their activities and to share the resources of the system, so that users perceive the system as a single, integrated computing facility.

Let us say about Google Web Server, from users perspective while they submit the searched query, they assume google web server as a single system. However, behind the curtain, google has built a lot of servers which is distributed (geographically and computationally) to give us the result within few seconds.

Advantages of Distributed Computing?

- ☐ Highly efficient
- ☐ Scalability
- ☐ Less tolerant to failures
- ☐ High Availability

Message Passing Interface (MPI) is a standardized and portable message-passing system developed for distributed and parallel computing. MPI provides parallel hardware vendors with a clearly defined base set of routines that can be efficiently implemented. As a result, hardware vendors can build upon this collection of standard low-level routines to create higher-level routines for the distributed-memory communication environment supplied with their parallel machines.

MPI gives user the flexibility of calling set of routines from C, C++, FORTRAN, and C #, Java or Python. The advantages of MPI over older message passing libraries are portability (because MPI has been implemented for almost every distributed memory architecture) and speed(because each implementation is in principle optimized for the hardware on which it runs)

Link for MPI Cluster Setup:

<https://medium.com/mpi-cluster-setup/mpi-clusters-within-a-lan-77168e0191b1>

MPI Installation

The machines which requires to be used to run MPI program, need to be connected using NIS and NFS. MPI can be installed by downloading the tar file from <https://www.mpich.org/> or using command

```
yum install mpich*
```

## Hello world code examples

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
// Initialize the MPI environment
MPI_Init(NULL, NULL);
// Get the number of processes
int world_size;
MPI_Comm_size(MPI_COMM_WORLD, &world_size);
// Get the rank of the process
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
// Get the name of the processor
char processor_name[MPI_MAX_PROCESSOR_NAME];
int name_len;
MPI_Get_processor_name(processor_name, &name_len);
// Print off a hello world message
printf("Hello world from processor %s, rank %d out of %d processors\n",
processor_name, world_rank, world_size);
// Finalize the MPI environment.
MPI_Finalize();
}
```

You will notice that the first step to building an MPI program is including the MPI header files with `#include <mpi.h>`.

After this, the MPI environment must be initialized with:

```
MPI_Init(int* argc, char*** argv)
```

During `MPI_Init`, all of MPI's global and internal variables are constructed.

For example, a communicator is formed around all of the processes that were spawned, and unique ranks are assigned to each process.

After `MPI_Init`, there are two main functions that are called. These two functions are used in almost every single MPI program that you will write.

```
MPI_Comm_size(MPI_Comm communicator, int* size)
```

`MPI_Comm_size` returns the size of a communicator.

In our example, `MPI_COMM_WORLD`

(which is constructed for us by MPI) encloses all of the processes in the job, so this call should return the amount of processes that were requested for the job.

```
MPI_Comm_rank(MPI_Comm communicator, int* rank)
```

MPI\_Comm\_rank returns the rank of a process in a communicator. Each process inside of a communicator is assigned an incremental rank starting from zero. The ranks of the processes are primarily used for identification purposes when sending and receiving messages.

A miscellaneous and less-used function in this program is:

MPI\_Get\_processor\_name(char\* name, int\* name\_length)

MPI\_Get\_processor\_name obtains the actual name of the processor on which the process is executing. The final call in this program is:

MPI\_Finalize()

MPI\_Finalize is used to clean up the MPI environment. No more MPI calls can be made after this one.

Compilation of MPI program

execute the following command to compile the program having MPI function calls

\$ mpicc <name of .c file>

Once compilation is successful. Execute the program as follows

\$mpitun -np <no. of processors><name of executable file>

\$mpirun -np 2 ./a.out ## np attribute initializes the MPI environment to let the program know how many processors are available to execute the program

Basic MPI data type

1. MPI data type C data type
2. MPI\_INT signed integer
3. MPI\_FLOAT float
4. MPI\_DOUBLE double
5. MPI\_CHAR signed char
6. MPI\_PACKED user-defined

The syntax of MPI\_Send is

|                              |               |
|------------------------------|---------------|
| int MPI_Send(void* msg_buf_p | /* input */,  |
| int msg_size                 | /* input */,  |
| MPI_Datatype msg_type        | /* input */,  |
| int dest                     | /* input */,  |
| int tag                      | /* input */,  |
| MPI_Comm communicator        | /* input */); |

The first three arguments determine the contents of the message; the last three determine the destination.

MPI\_Recv has a similar (complementary) syntax to MPI\_Send , but with a few important differences.

```
int MPI_Recv( void* msg_buf_p /* out */,
int buf_size /* in */,
MPI_Datatype buf_type /* in */,
int source /* in */,
int tag /* in */,
MPI_Comm communicator /* in */,
MPI_Status status_p /* out */);
```

Again, the first three arguments specify the memory available (buffer) for receiving the message.

The next three identify the message.

Both tag and communicator must match those sent by the sending process.

MPI program to send and receive data on processors

```
#include <stdio.h>
#include <string.h> /* For strlen */
#include <mpi.h> /* For MPI functions, etc */
const int MAX_STRING = 100;
int main(void) {
char greeting[MAX_STRING]; /* String storing message */
int comm_sz; /* Number of processes */
int my_rank; /* My process rank */
int i;
/* Start up MPI */
MPI_Init(NULL, NULL);
/* Get the number of processes */
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
/* Get my rank among all the processes */
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
if (my_rank != 0) /* Code to be executed on slave machines */
{
/* Create message */
sprintf(greeting, "Greetings from process %d of %d!", my_rank, comm_sz);
/* Send message to process 0 */
MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0,
0, MPI_COMM_WORLD);
```

```

}
else
{
/* Print my message */
printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
for (i = 1; i < comm_sz; i++) {
/* Receive message from process q */
MPI_Recv(greeting, MAX_STRING, MPI_CHAR, i, 0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
/* Print message from process i */
printf("%s \n", greeting);
}
}
/* Shut down MPI */
MPI_Finalize();
return 0;
}/* main end*/

```

### MPI Collective Communication Routines

MPI provides the following collective communication routines:

- ☐ MPI\_Barrier synchronization across all processes.
- ☐ MPI\_Broadcast from one process to all other processes.
- ☐ MPI\_Reduce: Global reduction operations such as sum, min, max, or user-defined reductions.
- ☐ MPI\_Gather : Gather data from all processes to one process.
- ☐ MPI\_Scatter : Scatter data from one process to all processes.
- ☐ Advanced operations in which all processes receive the same result from a gather, scatter, or reduction.
- ☐ There is also a vector variant of most collective operations where messages can have different sizes.

### Execute MPI program on different hosts

You specify the available hosts to MPI in three ways:

- ☐ Through the batch scheduler in your resource management software.  
By using a hostfile with the --hostfile option. The hostfile is a text file that contains the names of hosts, the number of available slots on each host, and the maximum slots on each host.
  - ☐ By using the --host option. Use this option to specify which hosts to include or exclude.
- Specifying Hosts By Using a Hostfile

The hostfile lists each node, the available number of slots, and the maximum number of slots on that node. For example, the following listing shows a simple hostfile:

```
node0
node1 slots=2
node2 slots=4 max_slots=4
node3 slots=4 max_slots=20
```

In this example file, node0 is a single-processor machine. node1 has two slots. node2 and node3 both have 4 slots, but the values of slots and max\_slots are the same (4) on node2. This disallows the processors on node2 from being oversubscribed. The four slots on node3 can be oversubscribed, up to a maximum of 20 processes. When you use this hostfile with the --nooversubscribe option mpirun assumes that the value of max\_slots for each node in the hostfile is the same as the value of slots for each node. It overrides the values for max\_slots set in the hostfile.

MPI assumes that the maximum number of slots you can specify is equal to infinity, unless explicitly specified. Resource managers also do not specify the maximum number of available slots.

#### Specifying Hosts By Using the --host Option

You can use the --host option to mpirun to specify the hosts you want to use on the command line in a comma-delimited list. For example, the following command directs mpirun to run a program called a.out on hosts a, b, and c:

```
% mpirun -np 3 --host a,b,c a.out
```

MPI assumes that the default number of slots on each host is one, unless you explicitly specify otherwise.

#### To Specify Multiple Slots Using the --host Option

To specify multiple slots with the -host option for each host repeat the host name on the command line for each slot you want to use. For example:

```
% mpirun -host node1,node1,node2,node2 ...
```



## **Practical Assignments:**

### **Set A**

- 1) Write an MPI program to calculates sum of randomly generated 1000 numbers (stored in array) on a cluster

### **Set B**

- 2) Write an MPI program to find the min and max number from randomly generated 1000 numbers (stored in array) on a cluster (Hint: Use MPI\_Reduce)

## **Assignment Evaluation**

0: Not Done [   ]

1: Incomplete [   ]

2: Late Complete [   ]

3: Needs Improvement [   ]

4: Complete [   ]

5: Well done [   ]

Signature of the instructor: \_\_\_\_\_ Date: \_\_\_\_\_